

AssertFlip: Reproducing Bugs via Inversion of LLM-Generated Passing Tests

Lara Khatib
David R. Cheriton School of
Computer Science
University of Waterloo
Waterloo, Ontario, Canada
l2khatib@uwaterloo.ca

Noble Saji Mathews
David R. Cheriton School of
Computer Science
University of Waterloo
Waterloo, Ontario, Canada
noblesaji.mathews@uwaterloo.ca

Meiyappan Nagappan
David R. Cheriton School of
Computer Science
University of Waterloo
Waterloo, Ontario, Canada
mei.nagappan@uwaterloo.ca

Abstract

Bug reproduction is critical in the software debugging and repair process, yet the majority of bugs in open-source and industrial settings lack executable tests to reproduce them at the time they are reported, making diagnosis and resolution more difficult and time-consuming. To address this challenge, we introduce ASSERTFLIP, a novel technique for automatically generating Bug Reproducible Tests (BRTs) using large language models (LLMs). Unlike existing methods that attempt direct generation of failing tests, ASSERTFLIP first generates passing tests on the buggy behaviour and then inverts these tests to fail when the bug is present. We hypothesize that LLMs are better at writing passing tests than ones that crash or fail on purpose. Our results show that ASSERTFLIP outperforms all known techniques in the leaderboard of SWT-Bench, a benchmark curated for BRTs. Specifically, ASSERTFLIP achieves a fail-to-pass success rate of 43.6% on the SWT-Bench-Verified subset.

CCS Concepts

• **Software and its engineering**; • **Computing methodologies**
→ *Artificial intelligence*;

Keywords

Bug Reproduction, Automated Test Generation, Large Language Models

ACM Reference Format:

Lara Khatib, Noble Saji Mathews, and Meiyappan Nagappan. 2026. AssertFlip: Reproducing Bugs via Inversion of LLM-Generated Passing Tests. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3744916.3773127>

1 Introduction

Bug reproduction is an essential first step in the software bug-fixing process [5], where software developers attempt to replicate the bug to observe the faulty behaviour and understand the root cause [22]. When a bug is discovered, a report is written in natural language and submitted to a bug-tracking system, containing relevant information about the issue. These reports often include a

detailed description of the issue, step-by-step reproducing instructions, observed vs. expected behaviour, software version details, and supporting materials such as screenshots and videos to help developers investigate the bug [31]. The reproduction steps outlined in the bug report can be converted into Bug Reproducible Test (BRT): a test case that fails when the bug is present and passes once the bug is fixed [6, 16]. Previous research has shown that developers often rely on BRTs to diagnose, debug, and fix bugs, in addition to ultimately verifying bug fixes [4]. However, despite their importance, many studies show that BRTs are rarely written at the time of bug reporting in open-source projects. Instead, they are usually added after the fix to validate that the bug has been resolved and will not reoccur [6]. Mundler et al. [16] found that in the SWE-Bench projects, no BRTs exist prior to a bug fix, and they are typically added as part of the pull request that introduces the fix. In the Defects4J dataset [9], only 4% of the bug reports include a failing test case [11]. Even in industrial settings, BRTs are usually deferred to the fix stage because bug reports often come from sources that lack the knowledge to create them at the time of reporting [6]. This implies that most bugs are reported without a reliable executable test, and developers are left with the task of reproducing the fault, which is time-consuming and challenging [21], and could delay bug fixing. Automatically turning bug reports into tests can make it easier to debug, validate, and fix issues, as well as reduce the time developers spend reproducing failures.

SWT-Bench [16], a benchmark for automated bug reproduction, demonstrates the growing potential of automatically generating BRTs. The current methods proposed for this task, which we discuss in more detail in the related work section (See Section 2), have yet to achieve strong results. Directly prompting an LLM creates BRTs successfully in only 3.6% of cases on SWT-Bench-Lite [16], a subset of SWT-Bench. Recent work shows that iterative prompting and multi-step interactions with LLMs can improve success rates on generation tasks [17, 20, 26, 27]. However, in the context of bug reproduction, two challenges remain: determining whether (a) the test has no implementation problems or bugs in the test code itself [1, 13, 17, 30] (b) the test fails for the right reason and exercises the bug [10, 17, 23]. To address this, we introduce ASSERTFLIP, a tool that generates bug-reproducing tests by first generating a test that passes on the buggy behaviour. If the test fails to run due to errors or setup issues, we refine it until it passes. Once we have a valid test that runs, we invert its logic to create a bug-revealing test. If the test cannot be fixed after a few rounds, we trigger a new regeneration attempt using the previous plan, test, and error to reflect on what went wrong in the earlier attempt. This pass-then-invert strategy



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICSE '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3773127>

can help avoid common failure modes in LLM-generated tests, such as broken syntax, setup errors, incomplete logic, or hallucinated assumptions.

Our key intuition is that LLMs are better at writing passing tests than at writing tests designed to crash or fail on purpose. We show that ASSERTFLIP outperforms prior methods at generating BRTs. Beyond these empirical gains, our findings point to a broader design paradigm for LLM-based bug reproduction. Prior work has treated failing-test generation as the default, but constraining the generation objective toward producing passing tests reshapes the entire workflow. Validation, coverage integration, and bug-report handling all need to be reconsidered under this premise. This shift defines a general design pattern of objective-driven generation, where explicit behavioral goals, such as “the test must pass” guide the structure of the workflow and open new directions for future research.

In summary, our main contributions of this paper are as follows:

- (1) We propose a novel pass-then-invert test generation technique that helps the LLM focus on writing correct tests before transforming them into BRTs.
- (2) We evaluate ASSERTFLIP on SWT-Bench and show that it outperforms prior work on fail-to-pass success rate.
- (3) We release our code and data to support reproducibility and to help others build on this work.

2 Related Work

SWT-Bench [16] is a recent benchmark developed specifically to evaluate the ability of LLMs to generate bug-reproducing tests. SWT-Bench has quickly become the standard benchmark for evaluating LLM-based bug reproduction, with many recent systems reporting results on it. It includes a public leaderboard on two subsets¹: SWT-Bench-Lite and SWT-Bench-Verified, the latter derived from SWE-Bench-Verified where human developers manually verified each instance to ensure a high-quality dataset [18]. The benchmark was introduced alongside evaluations of several LLM-based code agents like SWE-Agent [27], AutoCodeRover [29], and Aider [2], which were adapted with modified prompts for the task. The agents were evaluated on the fail-to-pass success rate, which is defined as the percentage of generated tests that fail on the buggy version of the code and pass after the corresponding fix is applied. This metric indicates whether a test correctly captures the buggy behaviour described in the issue and thus serves as a valid BRT. SWE-Agent+, a variant of SWE-Agent, achieved the highest fail-to-pass success rate reported in the SWT-Bench paper [16], with 19.2% on SWT-Bench-Lite, compared to 15.9% for SWE-Agent and just 3.6% for direct LLM prompting.

Amazon Q currently tops the leaderboard with 49% on SWT-Verified and 37.7% on SWT-Lite, though its architecture is undisclosed and does not rely on a single foundation model². Since Amazon Q does not disclose any details and includes minimal traces, we refrain from directly comparing to it. However, we do report it for completeness. OpenHands [24], an open-source developer-style agent, achieves 27.7% and 28.3% on the verified and lite subsets,

respectively, using Claude 3.5 Sonnet, despite not being specifically designed for bug reproduction.

A growing body of research has focused on LLM-driven methods for generating bug-reproducing tests from natural language issue descriptions. One of the earliest works in this area is LIBRO [10], which combines few-shot prompting, test post-processing, and heuristic ranking to generate BRTs. It was evaluated on the Defects4J and GHRB datasets [12]. In the SWT-Bench paper [16], LIBRO was adapted for Python and evaluated on SWT-Bench-Lite, achieving 14.1%. Otter [1] incorporates a self-reflective planner in which the LLM iteratively refines read/write/modify actions, then uses the final plan to guide test generation. Otter++ extends Otter by running multiple versions of the generation pipeline with different configurations and selecting the best test based on runtime feedback. This ensemble approach improves fail-to-pass rates on SWT-Verified from 31.4% to 37.0%, and on SWT-Lite from 25.4% to 29.0%. Issue2Test [17] introduces a three-phase pipeline that first uses meta-prompting to extract project-specific test-writing guidelines, then performs root cause analysis, and generates test candidates. It enters an execution-feedback loop with two LLM components, one to classify test failures and another to verify that assertion failures match the original bug report. On SWT-Bench-Lite, it achieves a 30.4% fail-to-pass rate. AEGIS [23] introduces a two-agent framework for bug reproduction where a searcher agent retrieves relevant context and a reproducer agent generates and refines test scripts. Its key contribution is a finite-state machine controller that guides the reproducer through structured feedback loops, including syntax checks, execution results, and external verification. They report results only on the Lite subset, but we exclude them from our comparison in line with Otter [1] due to unclear evaluation (they evaluate on SWE-Bench-Lite but compare against results from SWT-Bench-Lite, which is a different dataset).

Unlike the approaches mentioned above that attempt to generate failing tests and then attempt to analyze whether the failure is due to the intended bug or an unrelated issue, our approach starts by generating a passing test on the buggy version. If the test fails to run, we refine it until it executes successfully. We then invert its assertions to construct a BRT. ASSERTFLIP achieves the best results amongst all known approaches on the leaderboard.

3 Pass-first then invert

Other work in bug reproduction prompts the LLM to write a failing test that exposes the bug, often augmenting this process with a self-reflective planner [1], LLM-based validation [17], or execution and assertion matching loops [10]. We hypothesize that LLMs perform better when writing correct tests than when asked to create a test that *fails* on purpose [14]. We believe our method works primarily because we do not leave the responsibility of deciding when to stop and accept a test solely to the LLM. We adopt a more structured approach that allows us to better isolate the tasks of understanding the fault, creating a BRT, and validating the failure.

An LLM-generated test can fail for many reasons unrelated to the actual bug. These failures include a wide range of issues [7, 28], such as syntax errors, import errors, and unintended top-level execution (e.g., code running before the test starts). Tests may also fail due to non-self-contained logic, missing setup or teardown

¹<https://swtbench.com/?results=verified>

²<https://aws.amazon.com/q/>

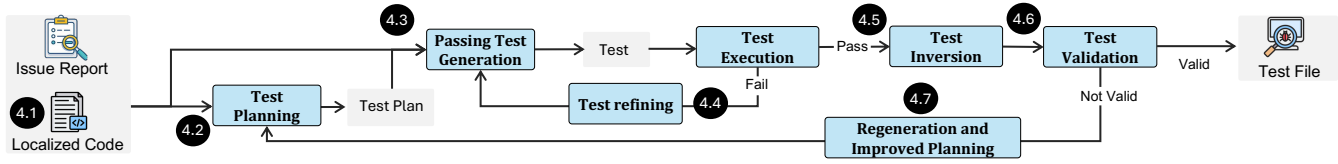


Figure 1: Overview of ASSERTFLIP pipeline.

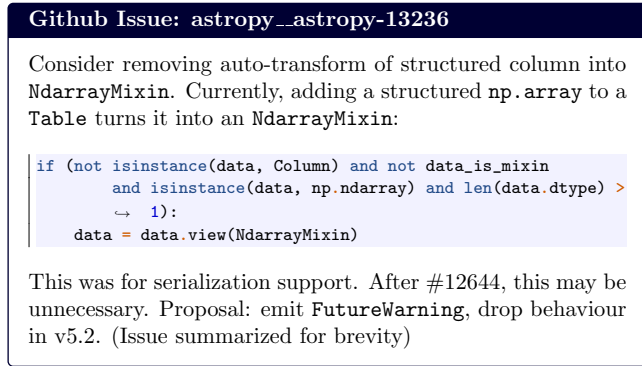


Figure 2: Bug report for astropy__astropy-13236 from SWT-Bench.

steps, or uninitialized variables. The LLM may introduce outdated or hallucinated APIs/classes/functions, misuse testing frameworks, or generate placeholder code (e.g., `assert False`, `# TODO`), or incomplete tests. Environment-level issues such as dependency mismatches, incorrect framework usage, or version mismatches can also cause a test to fail. If we rely solely on the LLM’s judgment to determine whether a test is valid and relevant to the bug, we risk accepting tests that fail for unrelated reasons, leading to false positives. While static analysis tools and linters such as `f1ake8` and `mypy` can help catch some of these issues, many deeper issues remain undetected.

To mitigate this, we adopt a more controlled approach. Specifically, we require that any candidate test to pass on the buggy version of the program before it is evaluated further. This ensures that we avoid common sources of test failure that are due to artifacts of code generation rather than the bug itself. Thus, we filter out invalid tests before invoking the LLM’s reasoning to assess bug relevance. This approach allows us to retain some control when using LLMs. It is easier and more reliable to confirm that a test passes under buggy conditions than to reason about the potentially numerous causes of failure. In doing so, we narrow the list of things that can go wrong and focus on one task: determining if a syntactically valid executable that passes the test indeed exercises the behaviour associated with the bug. Once a valid passing test is obtained, we invert its assertions to construct a BRT. Further details of our tool are provided in the next section.

4 ASSERTFLIP

This section describes ASSERTFLIP, an LLM-based tool that generates BRTs from natural language bug reports and source code. To direct the LLM toward writing a valid executable test that exposes

Example Test Plan (Pass-First)

- Test Setup:** Use numpy and the relevant parts of the astropy library.
- Test Input:** Create a structured np.array like:

```
np.array([(1, 2.0), (3, 4.0)], dtype=[('a', 'i4'), ('b', 'f4')])
```
- Bug Triggering:** Add the structured array to an astropy.Table. This causes a conversion to NdataArrayMixin.
- Test Structure:** Initialize the Table, add the array, and inspect the column type.
- Assertions:** Check that the column type is NdataArrayMixin. This is the incorrect behaviour but reflects the current bug.
- Edge Cases:** Try variants with different field types or empty structured arrays.
- Expected Outcome:** The test should pass for now, but will fail once the bug is fixed.

Figure 3: Test plan generated for astropy__astropy-13236.

the bug, we ask it to write a test that passes on the buggy behaviour and provide it with the issue report alongside the localized buggy code. We continue refining the test using execution feedback until it passes. Rather than trying to handle all the possible reasons an LLM-generated test might fail, we first ensure the test passes. Once we have a correct working test code that reflects the bug, we invert it so the test now fails when the bug is present. Our intuition is that LLMs are better at writing passing tests than at writing failing ones, where the failure could happen for any number of unrelated reasons. This approach contrasts with prior methods proposed for bug reproduction like Otter, LIBRO, and Issue2Test, which attempt to produce failing tests directly. Figure 1 depicts the full pipeline. We describe each stage below.

4.1 Localization input to the pipeline

As a first step, existing tools typically begin by localizing buggy code. Among the current approaches proposed for this task, most rely on LLMs for localization. For example, Otter [1] uses a four-step process that first localizes test files and functions by prompting the LLM to pick the top-10 relevant test files based on the issue description, then lets the LLM pick relevant test functions within those test files, and repeats the same process for localizing focal files and focal functions. Issue2Test [17] builds a hierarchical tree of the repository and provides it along with the issue report to the

LLM to rank the most relevant files. AEGIS [23] uses a Searcher Agent that employs an LLM to retrieve relevant code and test files based on the issue report using system commands and tool-specific interfaces.

Since the focus of this paper is on creating tests from issues rather than bug localization, we decided to assess our pipeline using localization output from an existing tool called Agentless. [25]. Agentless is a popular and, importantly, modular approach that we chose to obtain realistic localization data. The modular nature makes it possible to execute the localization phase alone, unlike most other tools in the space. For completeness, Agentless first identifies suspicious files by prompting a compact repository-structure representation and refining this selection through embedding-based retrieval. It then examines these files to isolate related code elements such as class declarations, function signatures, and variable definitions. Finally, it analyzes the actual code snippets of these elements to determine precise edit locations, which may be specified by line numbers, functions, or classes. Since Agentless generates multiple candidates at each stage, we adopt a conservative merging strategy that combines all proposed locations into a unified set.

4.2 Test Planning

Each run of ASSERTFLIP begins with a planning phase, where we pass the relevant code snippets and issue description obtained from the previous step to the LLM to generate a detailed test plan before writing any code. The goal of this step is to help the model reason about how to reproduce the bug described in the issue before attempting to write the test itself. The LLM is instructed to plan a test that will *pass* under the buggy version of the code, but still demonstrate the incorrect behaviour. Figure 3 shows a test plan generated by the LLM for a bug in astropy and its corresponding GitHub issue (see Figure 2), where structured np. arrays are incorrectly converted to NdataArrayMixin when added to a Table. We selected this issue at random as a running example to demonstrate our approach.

4.3 Passing Test Generation

The core of our system is the pass-invert method. Instead of asking the LLM to write a failing test from scratch, which may lead to broken or non-executable tests, we first instruct the model to generate a *passing* test that runs successfully but still exposes the bug. This test does not fail at runtime, but it reveals the incorrect behaviour described in the issue either through assertions that accept the buggy output or through comments explicitly identifying the flawed behaviour. Once the test is generated, we invert it by modifying its assertions such that it now fails when the bug is present and passes once the bug has been fixed. Using the plan from the previous step, we prompt the LLM to write a complete Python test using pytest or the testing framework used in the target project. We give it explicit constraints:

- The test must pass when run against the buggy version.
- The test must clearly show the bug by asserting the buggy behaviour or use `pytest.raises(...)` or equivalent logic for exceptions
- The code should be executable with no placeholders or syntax errors.

```
import pytest
import numpy as np
from astropy.table import Table
from astropy.table.ndarray_mixin import NdataArrayMixin

def test_structured_array_conversion_to_ndarraymixin():
    structured_array = np.array(
        [(1, 2.0), (3, 4.0)],
        dtype=[('a', 'i4'), ('b', 'f4')]
    )
    table = Table([structured_array])
    column_type = type(table.columns[0])
    # BUG: this is incorrect behaviour but currently happens
    assert column_type is NdataArrayMixin
```

Figure 4: The passing test generated by the LLM reflecting the buggy behavior.

Figure 4 shows the passing test that the LLM generated based on the aforementioned plan in Figure 3. This test passes, but only because the bug is still present, the conversion to NdataArrayMixin is happening, and the test explicitly checks for that.

4.4 Test Refinement Loop

Generated tests often fail to execute correctly on the first attempt. This could be due to a variety of reasons: missing imports, incorrect assumptions, setup errors, or subtle misunderstandings of the code. Rather than discarding these tests, ASSERTFLIP enters a *refinement loop*. In this loop, we prompt the LLM to revise the test based on the precise *error message* that occurred during execution. As long as the test is making progress (e.g., different errors are appearing), we continue refining it, up to a maximum of ten iterations. If a valid passing test is still not produced within this limit, the process moves to the regeneration and improved planning phase described in Section 4.7. This iterative strategy often allows the LLM to fix import paths, correct argument usage, or add missing test setup, without losing the structure of the originally generated test. An example of the prompt used during refinement is shown in Figure 5.

4.5 Inverting the Passing Test

Once we have generated a passing test that compiles, runs, and exercises code relevant to the bug, we flip its logic. This test already demonstrates the bug, as it passes because the system exhibits the buggy behaviour. At this point, we do not need to rewrite or redesign the test. Instead, we keep the structure the same and update the assertions to reflect the correct behaviour. We pass the issue report and prompt the LLM to do this with very specific instructions: change as little as possible, keep the test readable and minimal, and modify the logic so that the test fails when the bug is present and passes only when the bug is fixed. Any comments that previously acknowledged the buggy behaviour (e.g., `# BUG: this is incorrect`) are removed. Because the test is already valid and it compiles, executes cleanly, and covers the suspect lines, inversion becomes a simple step of changing assertions or outputs. Figure 6 shows the inverted version of the test for `astropy__astropy-13236`. This test now serves as a proper BRT: it fails on the buggy version of the code and will pass after the fix is applied.

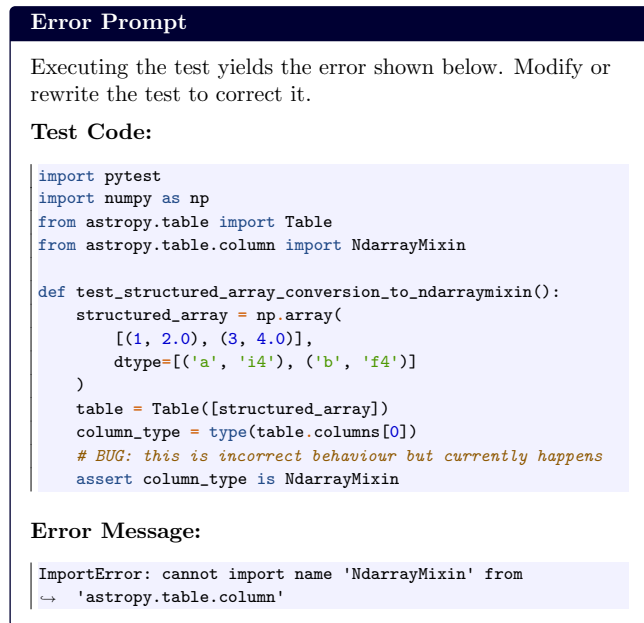


Figure 5: Error Prompt for astropy__astropy-13236

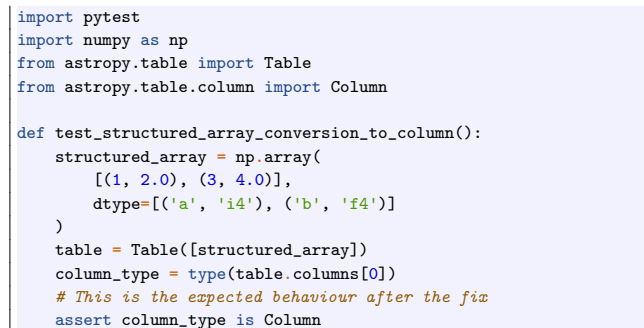


Figure 6: The inverted test now fails when the bug is present.

4.6 Test Validation

To ensure the test reveals the reported bug, we perform an LLM-based validation step. We run the test against the buggy version of the code, and pass the observed error trace along with the issue description into a separate LLM validation prompt shown in Figure 7. The validator is asked to determine whether the failure is caused by the bug described in the report or due to unrelated issues. If validation passes, the test is accepted. If the test fails validation, we trigger a new generation cycle, this time asking the LLM to rethink its plan entirely and try a new strategy for exposing the bug.

4.7 Regeneration and Improved Planning

If the system repeatedly generates tests that either fail or are rejected during validation, we do not allow the LLM to simply tweak the same test or revise its last response. Instead, we trigger a full **re-generation** phase. In this mode, the LLM is instructed to abandon its previous strategy and adopt a different approach. The idea of

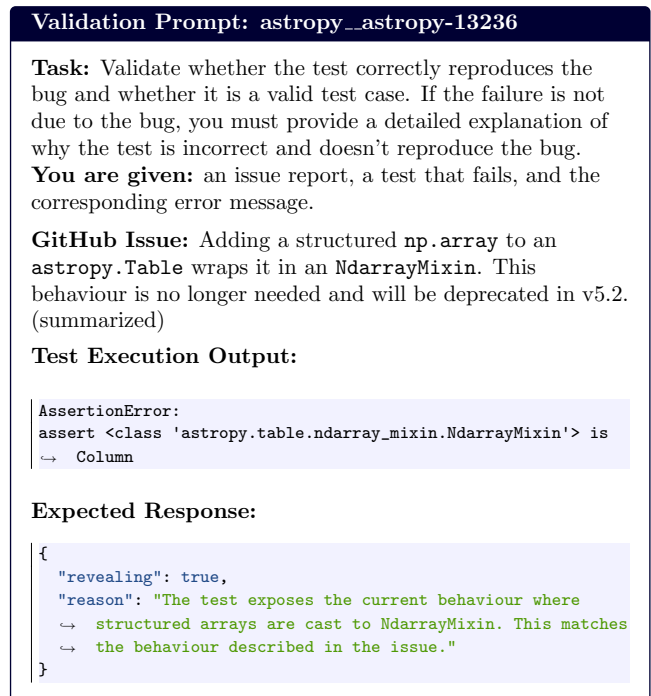


Figure 7: Prompt used to validate whether a test correctly reveals the bug.

this design is that if the LLM's initial reasoning is flawed, iterative refinements based on that flawed strategy are unlikely to yield valid results. To help it learn from its earlier mistakes, we include:

- The original bug report <ISSUE TICKET> and code snippets <CODE SNIPPETS>
- The previous plan <THOUGHT PROCESS>
- The failed test attempt <TEST ATTEMPT> and the error message <ERROR>
- The feedback on why the test was rejected <FEEDBACK>

The feedback is generated either during the validation step if the test does not correctly expose the bug or when the system fails to produce a passing test despite multiple refinement attempts, whether by repeatedly triggering the same error or exhausting the retry limit. The model is then prompted to rethink its strategy from scratch. We explicitly instruct the model not to reuse its earlier plan or structure. The prompt, shown in Figure 9, instructs the LLM to reflect on what went wrong in the previous attempt and generate a completely new plan that avoids the previous pitfalls [8]. This gives the system a second chance and encourages the LLM to diversify its reasoning and explore alternative test designs that might better expose the bug.

4.8 Additional Utilities: Code Retrieval via `get_info`

LLMs often struggle when working with partial code snippets, frequently hallucinating and making incorrect assumptions about missing code. This leads to errors during tasks like test generation or bug fixing. To reduce these errors, we introduce a tool function

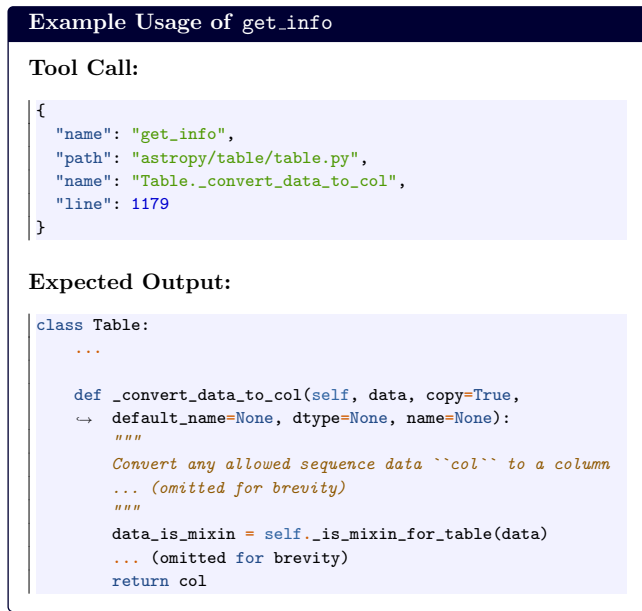


Figure 8: An example call to `get_info` for retrieving context.

that allows the LLM to request additional information about any names in the excerpt, such as functions, classes, or variables. The tool is implemented using OpenAI's function-calling interface [19]. At any point in the conversation, the model can request additional information about a symbol (such as code artifacts like functions or classes). The tool then performs static analysis to locate the symbol's definition and returns a trimmed, valid Python excerpt before continuing the conversation. To keep responses compact, less relevant sections are omitted and replaced with ellipses (...). The tool follows import paths and inheritance chains when needed and can merge context from multiple modules. This is especially useful in scenarios like refining a failing test. If the test references a function or variable with unclear behaviour, the tool can be used to provide enough context for generating a correct fix. Figure 8 shows an example of how the `get_info` tool is used to retrieve the definition of a method. This tool is available in the planning phases, test generation, and error fixing.

5 Evaluation

5.1 Experimental Setup

5.1.1 Benchmark. We use SWT-Bench [16] for our evaluation and run our tool on two datasets: SWT-Bench-Lite and SWT-Bench-Verified. Both are derived from real-world issue reports and patches in 12 popular open-source Python projects on GitHub. Each instance includes a natural language bug report, a corresponding fix patch, and a test that fails on the buggy version and passes after the fix is applied. The two datasets differ primarily in size and the strictness of their selection criteria. SWT-Bench-Lite contains 276 instances where only a single file is edited in the fix. SWT-Bench-Verified includes 433 instances that human developers have manually validated to ensure that each issue is clearly stated, the

Instructions:

You are an expert senior Python test-driven developer tasked with assisting your junior who is unable to write tests that reveal reported bugs. Your goal is to plan the creation of test functions that PASS but still expose the reported bug.

You will be provided with an ISSUE TICKET and a set of CODE SNIPPETS which might contain the buggy logic. You will also be given the THOUGHT PROCESS of your junior who was trying to write the test, the TEST ATTEMPT they wrote, the ERROR it produced. As well as FEEDBACK explaining why the previous test failed.

Your task is to analyze the described problem and previous attempt in detail and create a new PLAN for writing the test.

You MUST NOT reuse or copy the previous plan. You may use it to understand what failed, but you must take a different angle that avoids the same mistakes.

Figure 9: Instructions given in regeneration prompt.

patch is meaningful, and the test accurately reflects the fix, making it more reliable than the other subsets and thus our choice for focusing our evaluation on. We also include SWT-Bench-Lite in our evaluation because it is commonly used in prior work, which allows for easier comparison against the existing methods.

5.1.2 LLM. We use OpenAI's GPT-4o (gpt-4o-202408-06) for our experiments so that we can fairly compare against the other submissions on SWT-Bench, which utilize models from the GPT-4 family that have cutoffs before October 2023, when SWE-Bench (the underlying dataset behind SWT-Bench) was released.

5.1.3 Evaluation Metrics. To assess the effectiveness of our test generation approach, we adopt the evaluation metrics introduced by the SWT-Bench paper [16]. Specifically, we use two metrics:

- **Fail-to-Pass (F→P) Success Rate:** This metric measures the proportion of instances where at least one generated test fails on the buggy version but passes after the corresponding golden patch is applied. Such F→P tests are also used in prior work to indicate successful reproduction of issues and are also important in validating bug-fix correctness. A successful instance must contain at least one F→P test and no tests that fail after the patch (× → F).
- **Delta Mean Change Coverage (ΔC):** This metric specifically measures how well the generated tests cover the lines of code that were modified by the golden patch. It is computed as the percentage of modified (added, removed, or edited) lines that are newly covered by the generated tests.

5.1.4 Baselines. We evaluate ASSERTFLIP against systems reported on the SWT-Bench leaderboard for the Verified subset.³ This includes two baselines introduced by Mündler et al. [16]: *ZeroShotPlus*, which uses direct LLM prompting and generates a novel code diff

³<https://swtbench.com/?results=verified>

format introduced by their paper, and *LIBRO* [10], an earlier test generation system re-evaluated on their dataset and uses the same proposed patch. *Otter* [1] a recent approach for automated bug reproduction, and its variant *Otter++*, which selects outputs from five different prompting methods. Also listed is *OpenHands* [24], an open-source agent that was adapted for bug reproduction, and *Amazon Q* [3], a commercial system with results reported on the leaderboard, though its underlying setup is undisclosed. OpenHands uses Claude 3.5 Sonnet, and Amazon Q uses multiple foundation models, making their results not directly comparable to the other systems.

As for the Lite subset, we include results from five methods introduced in the SWT-Bench paper [16]: *ZeroShotPlus*, *LIBRO*, *AutoCodeRover*, *SWE-Agent*, and its variant *SWE-Agent+*. We also compare to prior works in the literature, such as *Otter*, its variant *Otter++*, and *Issue2Test*. Additional systems listed on the public leaderboard include *OpenHands* and *Amazon Q*.

5.2 Effectiveness of ASSERTFLIP against the Baselines

Table 1 reports the F→P rates and Δ Coverage scores for ASSERTFLIP and prior test generation systems on SWT-Bench-Verified. ASSERTFLIP achieves a 43.6% F→P success rate, successfully resolving 189 F→P tests out of 433 issues and outperforming all comparable baselines. Alongside its higher F→P rate, ASSERTFLIP also achieves higher Δ Coverage score, indicating that the generated tests exercise a larger portion of the buggy code compared to those from other systems. Following SWT-Bench, we define an instance as **resolved** if the *generated* test fails on the buggy version and passes on the fixed version. Otherwise, it is **unresolved**. We adopt this terminology throughout the evaluation.

While our reported score reflects performance over all 433 Verified issues, it is important to note that, unlike other systems, our approach deliberately abstains from generating solutions for every instance. We only generate 326 of the 433 cases. This is not a limitation but a feature, as our tool avoids producing low-confidence or misleading outputs, resulting in fewer incorrect responses that could waste developer time. This aligns with the motivation behind BouncerBench by Mathews et al. [15], which highlights the importance of abstention in automated software engineering systems, arguing that "sometimes no answer is better than a wrong one." We believe this enhances the trustworthiness of our system. As a result, our actual resolution rate is 58% (189 out of 326).

Among the baselines, ZeroShotPlus performs the lowest, resolving only 62 bugs. LIBRO shows a modest improvement at 17.8%, resolving 15 more bugs than direct prompting. Otter outperforms these techniques at 31.4% F→P rate, which increases to 37.0% with Otter++. The improvement comes from running the test generation stage five times with different heterogeneous prompts. Our approach, ASSERTFLIP, which generates passing tests and then inverts them, outperforms all these methods. This result supports the hypothesis that generating valid passing tests and flipping their oracle is more reliable than attempting to generate failing tests directly. Compared to Amazon Q, the current top-ranked system on the leaderboard, ASSERTFLIP performs closely, despite using only a single model. Amazon Q achieves a 49.0% F→P rate, representing a five percentage point advantage in success rate. However, direct

Approach	F→P		Δ Coverage (%)
	Total	Rate	
ZeroShotPlus (GPT-4o)	62	14.3	34.0
LIBRO (GPT-4o)	77	17.8	38.0
OpenHands* (Claude 3.5 Sonnet)	120	27.7	52.9
Otter (GPT-4o)	136	31.4	37.6
Otter++ (GPT-4o)	160	37.0	42.8
ASSERTFLIP (GPT-4o)	189	43.6	49.1
Amazon Q (Amazon Bedrock)*	212	49.0	57.4

Table 1: Comparison with prior methods on SWT-Bench-Verified. *Results not obtained under comparable settings.

Approach	F→P	
	Total	Rate
AutoCodeRover (GPT-4)	25	9.1
ZeroShotPlus (GPT-4)	28	10.1
LIBRO (GPT-4)	42	15.2
SWE-Agent (GPT-4)	46	16.7
SWE-Agent+ (GPT-4)	53	19.2
Otter (GPT-4o)	70	25.4
OpenHands (Claude 3.5 Sonnet)	78	28.3
Otter++ (GPT-4o)	80	28.9
Issue2Test (GPT-4o-mini)	84	30.4
ASSERTFLIP (GPT-4o)	99	36
Amazon Q (Amazon Bedrock)	104	37.7

Table 2: Comparison with prior methods on SWT-Bench-Lite.

comparison is limited since we know nothing about how Amazon's Q developer works.

Table 2 reports the results on SWT-Bench-Lite. ASSERTFLIP achieves a 36% F→P success rate, outperforming all other open and publicly described approaches. This includes Otter++, AutoCodeRover, and SWE-Agent+, which all use different architectures. Our method falls just one percentage point behind Amazon Q, the top-performing system on the public leaderboard. This result highlights the competitiveness of our pass-then-invert strategy.

5.3 Ablation Study

To understand the contribution of individual components in our tool, we conduct a series of ablation experiments. Tables 3 report F→P success rates under different configurations of the pipeline, including removal of the LLM-based validator, omission of the planning step, and the use of perfect localization. We also include a variant that retains the full pipeline but prompts the LLM to generate failing tests directly instead of following the pass-then-invert strategy. In this direct-fail variant, the validation stage is moved after test generation and execution to enable iterative refinement and ensure a fair comparison with the original pipeline. Table 4 shows variation in the number of regenerations.

The direct-fail variant represents one of the most important ablations demonstrating the effectiveness of our approach. Although

it uses the full pipeline, prompting the LLM to generate failing tests directly rather than passing tests that are then inverted, results in a substantial drop in performance, resolving only 105 instances compared to 189 in the pass-then-invert configuration. This corresponds to a decline of over 19 percentage points in $F \rightarrow P$ success rate, supporting our core intuition that LLMs are more reliable when asked to generate passing tests rather than failing tests directly.

Removing LLM validation results in a slight decrease in $F \rightarrow P$ success rate from 43.6% to 38.5%. Although there is a drop, we observe that the results are still superior to those of all comparable methods. While the number of generated tests increases from 326 to 352, the number of successful instances drops ($189 \rightarrow 167$), and the number of unresolved cases increases ($137 \rightarrow 185$). This suggests that the validation step plays a role in filtering and rejecting false positives that would otherwise be accepted. Moreover, when a test fails validation, it triggers a new regeneration cycle that often lead to successful test generation. Thus, the validation step is essential in filtering poor tests through iterative regeneration.

When removing the planner and re-planning phase, we observe an increase in the number of tests generated by our tool to 351. However, the number of successfully resolved instances drops to 181. This suggests that LLMs benefit from following a structured plan, which may help them stay on track and produce tests that are more likely to succeed. Although the drop in performance is relatively small, the overall accuracy of our tool, measured as the number of resolved instances out of the total generated, decreases from 58% to 51.6%. Since we care about reducing the generation of invalid tests, we retain the planner as it helps guide the LLM. Despite this, our system still outperforms all prior GPT-4o systems, further demonstrating the strength of our pass-then-invert technique.

We also evaluate the impact of localization quality by comparing performance under realistic (Agentless) and perfect localization. In our perfect localization setup, we use the Git patch from the fix in the SWT-Bench dataset [16] to localize the buggy code at both the file and line levels. To balance conciseness and completeness, we adopt a skeleton format: we treat each source file as an abstract syntax tree (AST) and collapse non-essential nodes, such that the edited lines are presented alongside their enclosing structures (classes, functions, blocks). This representation remains compact yet preserves the surrounding context. We observe that the total number of generated instances remains constant across localization settings, although the set of bugs that get resolved changes. Perfect localization resolves 24 instances that are unresolved under the realistic localization configuration. On the other hand, realistic localization successfully resolves 23 instances that remain unresolved under perfect localization. We also found 11 cases where tests were generated and resolved under perfect localization, but no tests were generated when using the realistic localization setup. These differences highlight how test generation is highly sensitive to localization quality and that different localization methods expose different sets of bugs. Importantly, even with realistic localization, which is much closer to what we would expect in real-world settings, our tool maintains strong performance and resolves a large number of instances. This suggests that the effectiveness of ASSERTFLIP does not depend on precise localization.

Table 4 shows how the number of regeneration attempts affects performance. Allowing more regenerations leads to an increase in

Variant	Generated Tests	$F \rightarrow P$	Rate (%)
ASSERTFLIP	326	189	43.6
Without LLM Validation	352	167	38.5
Without Planner	351	181	41.8
With Perfect Localization	326	189	43.6
Direct-Fail Variant	330	105	24.2

Table 3: Ablation study showing generation and $F \rightarrow P$ performance across system components.

Regenerations	Generated Tests	$F \rightarrow P$ Total	$F \rightarrow P$ Rate (%)
0	219	141	32.5
5	300	169	39.0
10 [†]	326	189	43.6

Table 4: Impact of regeneration attempts on generation and $F \rightarrow P$ success. [†] 10 regenerations is the default configuration.

the number of bugs resolved. Without any regenerations, the system solves 141 instances. This increases to 169 with five regenerations and 189 with 10, highlighting the value of re-planning as it gives the model multiple chances to expose the bug.

5.4 Cost Effectiveness of ASSERTFLIP

The average cost of running ASSERTFLIP per instance on SWT-Bench-Verified depends primarily on the number of regeneration attempts. At zero regenerations, the average cost is approximately 18 cents (0.1812 USD) per instance. This increases to 60 cents (0.6018 USD) with five regeneration attempts, and reaches 1.006 USD at ten regenerations. This includes all LLM interactions required throughout the entire pipeline: planning, test generation, test refinement, inversion, and validation. All computations use OpenAI’s GPT-4o pricing at the time of evaluation. The cost at the 10-regeneration setting, which is our default, remains similar to those reported for other unit test generation systems, bug reproduction tools, and LLM-based code agents. For users with cost constraints, running the system with five regeneration attempts offers a strong balance. The performance remains high and still outperforms all known tools on SWT-Bench-Verified, while cutting the cost by roughly 40% compared to the 10-regeneration setting. During generation, ASSERTFLIP accumulates context within each regeneration attempt to guide iterative improvements. Still, this context is discarded when a new regeneration cycle begins, and only the previous plan, test, and error message are passed. This design helps control prompt size and cost while preserving reasoning during each attempt.

We use the cost figures reported in the SWT-Bench’s paper [16], and compare them on the SWT-Bench-Lite subset, which consists of 276 instances. At 10 regeneration attempts, running ASSERTFLIP across all 276 instances would cost approximately \$266.7 and an average cost of \$0.96 per instance. In contrast, ZeroShot and ZeroShot-Plus cost around \$82, while LIBRO [10] costs \$420. SWE-Agent [27] and SWE-Agent+ are reported at \$290.71 and \$478.21, respectively. AutoCodeRover [29] is reported at \$368.4. These comparisons show

that even at its highest regeneration setting, ASSERTFLIP remains similar to or lower than most other approaches while delivering stronger performance.

To better understand where costs are incurred, we analyzed the total and per-instance cost of running ASSERTFLIP on all 433 verified instances from SWE-bench-Verified. The overall cost was \$435.92, averaging \$1.00 per instance. However, this average masks substantial variance across projects. For example, django dominated the cost profile, accounting for over 70% of the total (\$309.02), with an average of \$1.43 per instance across 216 instances. In contrast, projects like sympy and scikit-learn had much lower average costs per instance (as low as \$0.13). A detailed cost breakdown per project is provided in Table 5. This indicates that costs vary across projects and may depend on project-specific characteristics. To understand the source of cost variation, we initially examined factors such as fix difficulty, bug report length, and localization context, but found no clear correlation. A closer analysis of the execution traces later revealed that projects like django and sphinx, which showed the highest per-instance costs, have more project-specific mistakes and therefore more regenerations. We find that these two projects have more custom setups, like specific testing setup requirements, which LLMs might not be familiar with, leading to additional regenerations and higher overall cost.

Table 5: Per-project cost analysis from running ASSERTFLIP on 433 verified instances from SWT-Bench-Verified.

Project	Instances	Avg. Cost (USD)	Total Cost (USD)
django	216	1.43	309.02
sphinx	28	1.84	51.63
sympy	73	0.37	26.73
matplotlib	32	0.42	13.29
pytest	15	0.84	12.64
astropy	17	0.70	11.95
pylint	6	0.61	3.65
scikit-learn	24	0.13	3.17
xarray	15	0.16	2.47
requests	4	0.23	0.92
seaborn	2	0.20	0.39
flask	1	0.07	0.07
Total / Average	433	1.00	435.92

6 Discussion

6.1 Does fail-to-pass rate tell the whole story?

While the $F \rightarrow P$ rate is commonly used as the primary metric in bug reproduction benchmarks, it does not fully capture the distinct capabilities of different systems. In Figure 10, we visualize the overlap of resolved instances among ASSERTFLIP, Amazon Q, Otter++, and OpenHands on the SWT-Bench-Verified subset. ASSERTFLIP resolves 30 bugs that none of the other three systems handle. On the other hand, the other baselines each resolve instances that ASSERTFLIP misses. Amazon Q resolves 49 unique bugs, Otter++ resolves 11, and OpenHands resolves 15, with additional overlaps between them. The four systems collectively resolve 313 bugs, corresponding to a $F \rightarrow P$ rate of 72.2%, significantly surpassing the highest reported single-system $F \rightarrow P$ rate of 45% for Amazon Q.

This pattern indicates that these systems are often solving different types of issues, and that $F \rightarrow P$ rate alone can hide that. These

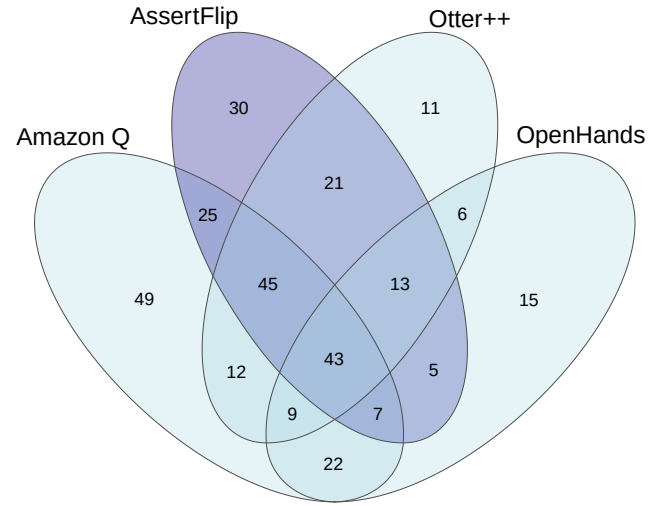


Figure 10: Overlap of resolved bugs across ASSERTFLIP Amazon Q, Otter++, and OpenHands on SWT-Bench-Verified.

differences may reflect variations in how each system processes bug reports, plans test strategies, or handles localization. The high number of resolved instances suggests that combining diverse methods, such as our pass-then-invert generation, multi-prompt ensembles, or the use of different models, could be more powerful than optimizing a single technique in isolation. As a result, future systems might benefit from hybrid approaches that leverage the complementary strengths of these tools.

6.2 How does issue fix difficulty impact test generation?

SWT-Bench-Verified includes difficulty annotations for each bug fix, categorized based on the estimated developer time to resolve the issue. We examine how our approach performs on bugs that are perceived to be difficult to fix because we believe reproducing these cases is especially valuable, as reproduction has been shown to significantly aid in resolving bugs [4]. Table 6 shows total instances resolved by difficulty level across four BRT tools. ASSERTFLIP performs competitively across all categories, resolving the most issues in the <15 min and 15min–1hr ranges and even demonstrating competitive performance on more complex 1–4 hour bugs. These findings highlight that ASSERTFLIP is not only effective on easy bugs but also performs well when tackling more complex issues.

Table 6: Comparison of systems across difficulty categories

Approach	<15 min fix (172 total)	15 min - 1 hour (225 total)	1–4 hours (36 total)
ASSERTFLIP	90	92	7
Amazon Q	91	111	10
Otter++	83	71	6
OpenHands	54	55	11

6.3 How does issue clarity impact test generation?

SWT-Bench-Verified is a curated dataset where professional developers manually reviewed each bug report. They removed any issues that were vague and underspecified. This means that each issue is clearly defined. However, real-world bug reports are often not so clean. Many are incomplete, poorly written, or confusing. To understand how this affects test generation, we look at what happens when we evaluate on SWT-Bench-Lite. SWT-Bench-Lite was designed as a smaller and more efficient version of SWT-Bench. It contains 276 tasks selected to reduce evaluation cost, be faster, and still cover a wide range of bugs. The authors of SWT-Bench-Lite employed automatic filtering to exclude issues with very short descriptions (fewer than 40 words) and multi-file edits. However, this filtering was not based on a manual review of issue quality. As a result, while SWT-Bench-Lite is intended to be easier to work with, it may still contain vague or unclear tickets.

Table 7 reports results on SWT-Bench-Lite, split by whether an issue also appears in SWT-Bench-Verified (overlap). We emphasize that this is not a direct comparison between the two datasets, rather it focuses on the Lite issues that are also present in Verified allowing us to isolate the effect of issue clarity under consistent task conditions. On the issues overlapping with the verified subset, ASSERTFLIP reaches a 45.5% F→P success rate. On the Lite-only (potentially vague) issues, this drops to 31.4%. This highlights how important clearly defined issue descriptions are for generating bug-revealing tests. Future work could involve handling vague or incomplete tickets, for example, by asking clarification questions or retrieving related context.

Table 7: ASSERTFLIP results on SWT-Bench-Lite split by whether the instance also appears in Verified.

Subset	Instances	F→P Total	F→P Rate (%)
Overlap with Verified	88	40	45.5
Lite-only	188	59	31.4
Total	276	99	36

6.4 Does coverage matter?

In our experiments, we noticed a consistent pattern across runs, whenever a bug was successfully resolved (F→P), the test that triggered it usually has high coverage over the lines modified by the bug fix. On the other hand, when the bug was not resolved, the test coverage over the patch was usually low. This was true even when components of the pipeline changed. This suggests that coverage over the buggy lines could be a useful signal for predicting whether a test is likely to be valid. Table 8 shows ΔC coverage results for our pipeline obtained using the official evaluation and reporting scripts from SWT-Bench [16]. The overall coverage delta across all instances is 49.1%, but when we break it down by whether the bug was resolved, we see a clear separation at 78.4% for resolved bugs vs. 26.1% for unresolved ones. This suggests that higher coverage of buggy lines is associated with better outcomes. Our pipeline uses an LLM-based validator to filter out tests that fail for the wrong reasons. However, this validator is not perfect, and in some cases it rejects tests that are correct or accepts ones that are not. Thus, it is

worth exploring whether we can improve the pipeline by combining LLM validation with coverage signals. One idea is to use coverage as a secondary filter. For example, we could reject any test that covers very few buggy lines. Another idea is to replace the LLM validation entirely with a threshold-based coverage check. However, coverage is only meaningful if the localization is accurate. When localization is noisy and the wrong files or lines are identified, a test with high coverage will not cover the correct code. This makes it harder to trust coverage as a signal in isolation. A promising direction is to combine both signals, use coverage to reject low-quality tests, and LLM validation to reason about correctness. We leave a full investigation of this idea to future work, but our early results suggest that coverage could be an effective component in filtering and validation.

Table 8: Δ Mean Change Coverage for ASSERTFLIP on SWT-Bench-Verified.

Metric	Value (%)
Coverage Delta (All) Δ^{all}	49.1
Coverage Delta (Resolved) Δ^S	78.4
Coverage Delta (Unresolved) $\Delta^{\text{not } S}$	26.1

7 Threats to validity

Our experiments rely on the SWT-Bench Lite and Verified datasets. These benchmarks are designed to evaluate BRTs, but they do not represent all types of bugs or all codebases. The test cases are derived from a small set of Python projects, which means the results may not apply to other languages or less common frameworks. While our approach is conceptually language-agnostic, extending it beyond Python would require replacing the test runner and adapting the prompt templates to the target language’s testing framework and syntax conventions. We have not yet evaluated it with other languages, and we therefore include this as a threat to validity.

A major threat is the unknown overlap between benchmark data and the training data of the LLMs we use. Models like GPT-4o are trained on data that is not publicly disclosed, so it is possible some of the benchmark code or bug reports were seen during training. We cannot fully control for this. This is a limitation for all prior work using closed LLMs [1, 10, 16, 17, 23]. We used GPT-4o for all test generation and validation in this study. While we acknowledge that the performance of ASSERTFLIP may vary if a different LLM is used, this choice was made to keep the focus on the improvements derived from the approach itself, disregarding advances in LLMs themselves. Furthermore, this also lets us steer away from concerns of data leakage since the model has an early knowledge cutoff of October 2023. Prompting is a key part of our workflow. To reduce variation, we use a fixed prompt structure in all experiments and tune only a small set of parameters. All ablations keep prompts consistent except for the tested change. The prompts, regeneration limits, and run settings were chosen iteratively, not through exhaustive search. Bug localization is another key factor. We do not focus on localization itself in this study, and our main results utilize localization from Agentless [25]. In real projects, localization may be less accurate. Further, since only code localization was available to us from past work, we do not perform test localization. Our design choices, however, allow us to create standalone tests that can be

added to the test suite without concerns of modifying existing tests and can keep the BRTs well isolated.

LLMs are inherently non-deterministic, and stochastic behavior is a known limitation across all LLM-based approaches. To minimize randomness, we set the sampling temperature to zero in all our experiments. Furthermore, we randomly selected ten instances from the full set using a uniform sampling procedure with a fixed seed, and ran each of those instances five times under the same configuration. This was done to provide transparency around expected variance while staying within our cost constraints. Five of the ten instances produced identical outcomes across all runs. Three instances were consistent in four out of five runs while the remaining two instances in three of the five runs. These results show that the tool behaves reliably across repeated executions with limited variation that is expected in systems driven by LLMs. Our analysis in the paper attempts to present a holistic picture of the effectiveness of our technique on established benchmarks and does not focus on the outcome of specific instances. This interpretation is consistent with prior work in this space.

8 Conclusion

In this paper, we introduce ASSERTFLIP a novel approach for automated bug reproduction from issue reports. We evaluate ASSERTFLIP on SWT-Bench and find that it outperforms all the known approaches on the leaderboard. ASSERTFLIP achieves a 43.6% success rate on the SWT-Bench-Verified subset, making it the most effective open tool for this task. This performance validates the strength of our Pass-then-Invert strategy. Future work could explore combining ASSERTFLIP with complementary approaches to better leverage their strengths, look into handling incomplete or vague bug reports, and integrate coverage metrics into the validation process. ASSERTFLIP is a promising step toward automating the bug reproduction process, helping enable faster and more efficient debugging and repair workflows.

9 Data Availability

Our code, prompts, and full experimental results are available at the following online repository: <https://github.com/uw-swag/AssertFlip>

References

- [1] Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. 2025. Otter: Generating Tests from Issues to Validate SWE Patches. *arXiv preprint arXiv:2502.05368* (2025).
- [2] Aider. 2025. Aider. Retrieved June 30, 2025 from <https://aider.chat/>
- [3] Amazon Web Services. 2024. Amazon Q. <https://aws.amazon.com/q/>. Accessed: July 8, 2025.
- [4] Moritz Beller, Niels Spruit, Diomidis Spinellis, and Andy Zaidman. 2018. On the dichotomy of debugging behavior among programmers. In *Proceedings of the 40th International Conference on Software Engineering*. 572–583.
- [5] Oscar Chaparro, Carlos Bernal-Cárdenas, Jing Lu, Kevin Moran, Andrian Marcus, Massimiliano Di Penta, Denys Poshyvanyk, and Vincent Ng. 2019. Assessing the quality of the steps to reproduce in bug reports. In *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 86–96.
- [6] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic Bug Reproduction for Effective Automated Program Repair at Google. *arXiv preprint arXiv:2502.01821* (2025).
- [7] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Weikang Zhou, Muling Wu, Mingxu Chai, Jessica Fan, Caishuang Huang, Yunbo Tao, et al. 2024. What's wrong with your code generated by large language models? an extensive study. *arXiv preprint arXiv:2407.06153* (2024).
- [8] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. Critic: Large language models can self-correct with tool-interactive critiquing. URL <https://arxiv.org/abs/2305.11738> (2024).
- [9] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440.
- [10] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323.
- [11] Anil Koyuncu, Kui Liu, Tegawendé Bissyandé, Dongsun Kim, Martin Monperrus, Jacques Klein, and Yves Le Traon. 2019. iFixR: Bug report driven program repair. In *Proceedings of the 2019 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 314–325.
- [12] Jae Yong Lee, Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2024. The github recent bugs dataset for evaluating llm-based debugging applications. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 442–444.
- [13] Yalan Lin, Yingwei Ma, Rongyu Cao, Binhua Li, Fei Huang, Xiaodong Gu, and Yongbin Li. 2024. LLMs as continuous learners: Improving the reproduction of defective code in software issues. *arXiv preprint arXiv:2411.13941* (2024).
- [14] Noble Saji Mathews and Meiyappan Nagappan. 2024. Design choices made by llm-based test generators prevent them from finding bugs. *arXiv preprint arXiv:2412.14137* (2024).
- [15] Noble Saji Mathews and Meiyappan Nagappan. 2025. Is Your Automated Software Engineer Trustworthy? *arXiv preprint arXiv:2506.17812* (2025).
- [16] Niels Münder, Mark Müller, Jingxuan He, and Martin Vechev. 2024. SWT-bench: Testing and validating real-world bug-fixes with code agents. *Advances in Neural Information Processing Systems* 37 (2024), 81857–81887.
- [17] Noor Nashid, Islem Bouzenia, Michael Pradel, and Ali Mesbah. 2025. Issue2Test: Generating Reproducing Test Cases from Issue Reports. *arXiv preprint arXiv:2503.16320* (2025).
- [18] OpenAI. 2024. Introducing SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>. Updated: February 24, 2025; Accessed: 2025-06-30.
- [19] OpenAI. 2025. *Function Calling*. <https://platform.openai.com/docs/guides/function-calling?api-mode=responses> Accessed July 14, 2025.
- [20] Juan Altmayer Pizzorno and Emery D Berger. 2024. Coverup: Coverage-guided llm-based test generation. *arXiv preprint arXiv:2403.16218* (2024).
- [21] Philipp Straubinger and Gordon Fraser. 2023. A survey on what developers think about testing. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 80–90.
- [22] Dhaval Vyas, Thomas Fritz, and David Shepherd. 2014. Bug reproduction: A collaborative practice within software maintenance activities. In *COOP 2014-Proceedings of the 11th International Conference on the Design of Cooperative Systems, 27-30 May 2014, Nice (France)*. Springer, 189–207.
- [23] Xinchun Wang, Pengfei Gao, Xiangxin Meng, Chao Peng, Ruida Hu, Yun Lin, and Cuiyun Gao. 2024. AEGIS: An Agent-based Framework for General Bug Reproduction from Issue Descriptions. *arXiv preprint arXiv:2411.18015* (2024).
- [24] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. [n. d.]. OpenHands: An open platform for AI software developers as generalist agents, 2024b. URL <https://arxiv.org/abs/2407.16741> 2, 4 ([n. d.]), 9.
- [25] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* (2024).
- [26] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using chatgpt. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 819–831.
- [27] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
- [28] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1703–1726.
- [29] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604.
- [30] Zhihao Zhu, Yi Yang, and Defu Lian. 2024. TDDBench: A Benchmark for Training data detection. *arXiv preprint arXiv:2411.03363* (2024).
- [31] Thomas Zimmermann, Rahul Premraj, Nicolas Bettenburg, Sascha Just, Adrian Schroter, and Cathrin Weiss. 2010. What makes a good bug report? *IEEE Transactions on Software Engineering* 36, 5 (2010), 618–643.